TECHNICAL SOLUTION DOCUMENT

# Deepfake and Manipulated Media Detection

A triage service for video, image and audio. Built to be auditable, and honest about what it does not know.

| Scope | Deployment | Interface | Status |
|---|---|---|---|
| MVP, three media types | Docker Compose, single GPU node | Web application | Active build |

## 1 Understanding the Problem

Fake and edited media is cheap to make now. Face swaps, puppeted video, cloned voices. It also spreads fast. On social platforms most of the damage happens in the first few hours, long before anyone can check a clip by hand. The task is to build something that helps catch this content early.

Reading past the surface of that, four things shape everything below.

**The output is a signal, not a verdict.** No detector is right all the time, and accuracy drops sharply on media the model was never trained on. A system that confidently says "FAKE" is more dangerous than useful, because a false positive on real footage gets a real person blamed for something they did not do. So the honest output is a score, a band, and a clearly marked "not sure" range. Not a yes or no label.

**Every upload is hostile.** Anyone can send anything. The file gets opened by media decoders, and those have a long history of memory safety bugs. On top of that, the scores themselves can be pushed around by an attacker who tweaks the input in small ways. Both of these are designed around. Neither is assumed away.

**The media is biometric.** What gets uploaded here is faces and voices. Keeping that forever grows the damage of any future breach, and grows the legal exposure with it. So deletion has to be the default and it has to be provable, while enough evidence survives that a disputed result can still be looked at again.

**A result nobody can reconstruct is worthless.** If someone challenges a decision weeks later, "the model said 84" is not good enough. The record has to say which exact weights ran, how the per item scores were combined, what settings were used, and a hash of the bytes that were analysed. All of it written at the moment the decision is made, not afterwards.

**What this does not do**

This system does not decide whether something is true, and it does not decide whether anything should be taken down. It looks for signs that media was synthesised or edited, and routes the result. Whether a video is authentic and whether it is honest are two different questions.

## 2 The Proposed Solution

A service that takes a video, image or audio file and returns three things. A calibrated score for how likely the media is to be manipulated, a band that decides what happens next, and an audit record that makes the result reproducible. The uploaded media is deleted on a timer by default.

### 2.1 Three pipelines, one place where decisions happen

Each type of media is handled by a pipeline suited to it. All three then meet at the same aggregation, the same band routing and the same retention logic. That keeps one place where a number turns into a decision, instead of three slightly different ones that drift apart over time.

### 2.2 Score the pieces, then combine them carefully

A model scores individual frames, faces or audio chunks. Those scores are then combined using a confidence weighted trimmed mean. Never a plain average, because a handful of badly cropped faces can drag a plain average anywhere they like. Items are also weighted down by how confident the face detection was, so a frame where the face was barely found counts for less.

**Two of those three mechanisms have never actually fired.** The confidence weighting is real and does change results. The trimming and the low confidence dropping have not: across 378 scored items the lowest confidence ever produced was 0.6, so the 0.3 cutoff has never dropped anything, and trimming 10 percent of a six item job rounds down to nothing, which is most jobs. So what has run so far is a weighted mean, not a trimmed one. The settings are untuned defaults that look like tuned ones, and they cannot be tuned until real weights produce a real spread of scores and confidences. This is written down rather than left for someone to discover.

### 2.3 Refuse to guess

If face detection finds nothing at all, the result is `undetermined`. It is never quietly turned into "authentic". If there is more than one face, each one is scored separately and the frame takes the worst of them, so a single edited face in a crowd does not get averaged into nothing.

### 2.4 Bands, not raw percentages

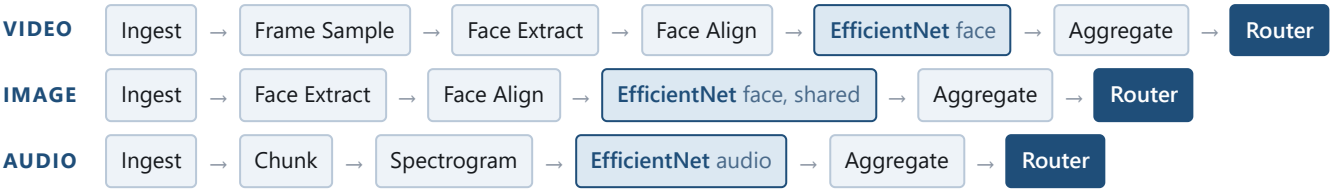
The combined score maps onto a band, and the band decides what happens. The bands cover the whole range with no gaps, so there is never an input the router does not know how to handle.

| SCORE | BAND | WHAT HAPPENS |
|---|---|---|
| under 20 | Likely authentic | Cleared automatically, deleted on schedule |
| 20 to 40 | Leaning authentic | Cleared automatically, deleted on schedule |
| 40 to 60 | Uncertain | Flagged for review, alert sent |
| 60 to 80 | Leaning manipulated | Flagged at low urgency, still deleted on schedule |
| over 80 | Likely manipulated | Flagged and alerted, extended retention window opens |
| none | Undetermined | No usable signal, flagged, never scored as real or fake |

The 60 to 80 range gets special care. It sits right below the most serious threshold, and the calibration around that threshold is not trusted enough to show anyone a raw percentage. So results there are logged and flagged rather than quietly passed through to normal deletion.

## 3 How the System is Built

### 3.1 The detection pipelines




The face model is shared by video and image, so both carry one `model_version_id`. Audio uses a separate model with its own. For images there is a single item to score, so aggregation is the identity function.

### 3.2 Services, and the trust boundary between them

The network layout here is a security control, not tidiness. The container that opens attacker supplied media has no route off the machine at all.

**EDGE, REACHABLE FROM THE HOST**

- **Browser**
- **Gateway**
  FastAPI on port 8000
- **Object storage**
  MinIO or S3 on port 9000, signed POST upload

**INTERNAL, NO ROUTE OFF THE HOST**

- **CPU preprocess**
  decodes untrusted media, read only, all capabilities dropped
- **GPU inference**
  EfficientNet on CUDA
- **Router**
  bands and retention
- **PostgreSQL and Redis**
  job row is the audit trail, queue and live status
- **Cold storage and sweeper**
  extended window copies, crash recovery, expiry

No virus scanning is deployed. Container isolation and lockdown stand in for it, and they ship alongside the worker that parses untrusted input rather than arriving later.

### 3.3 What happens to a request

1. The client asks for a job. The gateway rate limits the request, creates the database row, and hands back a signed upload grant.
2. The client uploads the file straight to object storage. Large media never goes through the API at all.
3. The client tells the gateway it is done, and the job is queued. Calling this twice is safe and does not queue the job twice.
4. The CPU worker decodes the file, samples frames, and finds and aligns faces. For audio it cuts the file into chunks and turns them into spectrograms.
5. The GPU worker scores every item and writes a row for each one.
6. The router combines the scores, picks a band, raises any flags, and starts the deletion of the working media.
7. Status updates are pushed over a WebSocket the whole time, with polling as a fallback if the connection drops.

## 4 Key Features

### 4.1 Deletion you can actually check TIER 1

Uploads and the face crops made from them are deleted once inference finishes. Every path that can delete something checks a hold flag first. That includes the normal delete, the recovery sweep that catches jobs a crashed worker left behind, and the cold storage timer. The flag column and the check that reads it are written in the same commit as the delete code, before anything can set the flag.

### 4.2 The job row is the audit trail TIER 1

Every result stores a SHA 256 hash of the bytes that were analysed, the id of the model that produced the scores, and the exact method and settings used to combine them. The per item scores are kept after the media is deleted, because they are just numbers rather than biometric data. So a disputed result can still be examined without keeping a single face on disk.

### 4.3 An upload path that cannot be used as free storage TIER 1

Uploads go straight to storage by design, which means the API never sees those bytes and cannot check their size. So the upload grant carries a signed size limit that object storage enforces itself. The simpler kind of signed URL cannot express a size limit at all, which would have left the bucket open to unlimited writes.

### 4.4 The extended retention window TIER 2

When a result lands in the highest band, the face crops that actually drove that score are copied into cold storage on a fixed 30 day timer. Not the whole original file. That way a dispute has the specific evidence to look at, while the upload itself is still deleted on the normal schedule.

> **This is not a legal hold**
>
> It is a fixed timer that expires by itself, and it is described that way everywhere in the code, in the API responses and in this document. A timer that quietly runs out in the middle of a dispute is a liability dressed up as a safeguard. Relying on it for a real dispute needs legal sign off, which this codebase does not claim to have.

### 4.5 Isolation instead of virus scanning TIER 3 SUBSTITUTE

The worker that opens uploaded media has no antivirus in front of it. Instead it runs on an internal only network with no route off the machine, a read only filesystem, every Linux capability dropped, no ability to gain new privileges, a temporary directory that cannot execute anything, and hard limits on memory and process count.

### 4.6 Failing in ways you can see

- **Dead letter queue.** A job retries a limited number of times, then stops and is recorded with the reason. It is never silently lost, and never retried forever.
- **Durable status.** Redis runs with persistence turned on, and the service refuses to start without it, so a restart cannot wipe out the status of jobs that are still running.
- **Recovery sweeps.** Background sweeps find jobs that finished, or failed, or were abandoned, and clean up any media they left behind. Every one of those sweeps respects the hold flag, and a sweep asks the queue whether work is still waiting before it touches anything.

### 4.7 Every result says how far it can be trusted

Results carry plain warnings attached to them. One says the score can be pushed around by an attacker editing the input. Another says how trustworthy the model itself is. That second one fails safe on purpose. If the system cannot tell which model produced a score, it gives the strongest warning rather than staying quiet.

## 5 Technology Choices

| LAYER | CHOICE | WHY |
|---|---|---|
| API | Python 3.11, FastAPI, Uvicorn | Async support for pushing live status over WebSockets, and typed request and response models. |
| Models | PyTorch, EfficientNet | Good accuracy for the compute cost. Separate models for faces and for audio, each calibrated on its own. |
| Calibration | Temperature scaling | Cheap, and only needs a held out set. It is a snapshot taken at launch, nothing more. |
| Database | PostgreSQL 16 | Transactional audit trail. JSON columns for the aggregation settings. Partial indexes keep the cleanup sweeps fast. |
| Queue and status | Redis 7 with consumer groups | One dependency covers both the work queue and live status. If a worker dies holding a job, another one can pick it up without waiting for a restart. |
| Object storage | MinIO locally, S3 when deployed | Same upload behaviour in both, so nothing changes between them. MinIO simply drops out of the setup at deploy time. |
| Packaging | Docker and Docker Compose | One image per service, and the network isolation is written down rather than assumed. |
| GPU | NVIDIA CUDA through WSL2 | Checked on an RTX 4060 with 8 GB before any model code was wired in. |
| Testing | pytest, plus live checks | 246 automated tests passing, plus scripts that check the running system against a real database, real Redis and a real bucket. The database methods are run against a stand in shaped like the real driver, so a call the driver does not support fails here rather than in production. |
| Supply chain | gitleaks | A secrets scan over the full history. Run on 29 August 2026 across all 30 commits, no findings. This row used to say the scan runs before anything is pushed publicly, which was not true at the time: the scan had never been run and most of the history was already public. |

**Why not Kubernetes yet**

Setting up a cluster, secrets, ingress and storage volumes is days of work on its own, and on a single GPU machine it buys nothing that Compose does not already do. Compose runs the same set of services, so moving later is a packaging change rather than a redesign. The reason to move would be a second machine, a real uptime requirement, or needing to scale the GPU workers. Having spare time is not by itself a reason.

## 6 Implementation Plan

The plan is ordered by dependency, not by calendar. Each phase is there because the one after it cannot be built safely until it exists. The rule running through all of it is that harder work gets deferred in clearly named tiers, so nothing is ever skipped quietly. Something set aside is written down as set aside, not left out and forgotten.

### 6.1 How the work is ordered

| PHASE | FOCUS | WHAT IT COVERS |
|---|---|---|
| **Phase 1** | Skeleton and contracts | The database schema including the hold flag, configuration, the storage layer, queue and retry behaviour, and rate limiting on the way in. Nothing scores anything yet. The shape of a job and the rules around it have to exist before anything starts writing to them. |
| **Phase 2** | Ingest and isolation | Signed uploads, the API endpoints, and live status. The worker that opens uploaded media ships together with its container lockdown, in this same phase, because a worker parsing untrusted input before isolation exists is an open door. |
| **Phase 3** | Scoring and routing | Inference behind a swappable backend, so the pipeline can be built and proven before real weights arrive. Score combining, worst case rollup when a frame holds more than one face, band routing, and review flags. |
| **Phase 4** | Retention | Timed deletion, the hold flag check on every path that can delete, the extended retention window for the highest band, and the recovery sweeps that clean up after a crashed worker. |
| **Phase 5** | Proof it works | The whole stack running together and checked against a real database, real Redis and a real bucket, with GPU access verified before any model code depends on it. |
| **Phase 6** | Real weights | Load the face and audio checkpoints, run calibration separately for each model, then check the score bands against real score distributions instead of placeholder output. |

### 6.2 What is deferred on purpose

- **Tier 1, not negotiable.** Deletion with the hold flag checked on every delete path, signed uploads with a size limit that storage enforces, rate limiting at the entrance, the job row as the audit trail, and job status that survives a restart.
- **Tier 2, stubbed rather than skipped.** The extended retention window, calibration as a launch snapshot rather than a retraining pipeline, and a dead letter queue that logs rather than pages anyone.
- **Tier 3, deferred and written down.** The adversarial input classifier, a human review dashboard, and virus scanning. Each one has either a named substitute or a named gap. A half built adversarial classifier would be worse than an honestly absent one, because it would look like protection.

### 6.3 What makes this approach different

- **It says when it does not know.** Undetermined is a real outcome with its own path through the system, and the uncertain band is shown as uncertain rather than rounded into a verdict. Most detectors hand back a number and leave the reader to guess how much of it to believe.
- **The audit trail is the product, not logging.** The hash, the model id, the combination method and its settings are written onto the job row at the moment the decision is made. A result that cannot be reconstructed later counts as a broken result.
- **Deletion is proven against storage, not against a flag.** The check is whether the object is really gone from the bucket, on every path that can delete, including the ones that run after a crash.

- **The trust warning fails closed.** If the system cannot work out which model produced a score, it gives the strongest warning rather than none. The dangerous version of that check is the one that goes quiet exactly when the scores start to look believable.
- **The risky code is boxed in, and the box ships with it.** The only component that opens attacker supplied bytes has no route off the machine, and that isolation belongs to the same phase as the component itself.
- **Claims about the system carry their evidence.** Every statement about how a piece of running software behaves records whether someone ran it, read it in a vendor document, or worked it out by reasoning. Reasoning that reads like measurement is how wrong beliefs survive, so the difference gets written down at the time rather than guessed later from how confident the sentence sounds.
- **A test has to be seen failing first.** Any test written to catch a specific bug is watched going red before the fix goes in, against the real system rather than a stand in. Several tests here passed while never once observing the thing their name claimed, which is what turned this into a rule.

### 6.4 Limitations, stated plainly

> **What this system does not claim**
>
> - **It is not robust against attackers.** Someone who knows what they are doing can edit an input to move the score. This is written down rather than hidden, and every result says so.
> - **There is no legal hold.** The extended retention window is a fixed timer that expires on its own.
> - **The calibration is not production validated.** It is a snapshot taken at launch and nothing keeps it up to date.
> - **No compliance claim is made.** Scoped deletion and a partial audit trail are much better than nothing, but whether that meets any particular regulation is a legal judgement this codebase does not get to make.
> - **It is not a detector yet.** The default backend derives a score from a hash of the input. It exists so the pipeline can be built and proven before real weights arrive. It carries no detection meaning, and it says so on every result it produces.
> - **Two of the robustness settings are untuned.** The confidence floor and the trimming fraction in the aggregation have never changed a result on the data seen so far. Confidence weighting genuinely happens, but trimming and dropping have not yet fired, and neither can be tuned until real weights produce a real spread of scores.

That last group is the part most worth being clear about. What the phases above build is a *system* that works from end to end. Ingest, scoring, combining, routing, deletion and the audit trail. What they do not build is *detection accuracy*, because that waits on weights. Treating those two as the same thing would be the most misleading thing this document could do.